```
> +4
[1] 4

> 5 * 3
[1] 15
```

Examples of a few built-in mathematical functions that you can use from the R console are given below:

```
> cos(90)
[1] -0.4480736

> sin(90)
[1] 0.8939967

> exp(2)
[1] 7.389056

> log(10, 2)
[1] 3.321928
```

You can define a new function using the *function* keyword, followed by named arguments and the body of the function. For example:

```
> f <- function (a, b) { a * b }

> f(2, 3)
[1] 6
```

You can also pass functions as arguments to other functions. In the following example, *proc* is a function that takes two arguments, a value and a function, applies the value to the function and returns the result.

```
> proc <- function (x, func) {
+   return(func(x))
+ }

> proc(90, sin)
[1] 0.8939967
```

If you only want to create a local function, you can create an anonymous function in R as follows:

```
> (function(x) x * 2) (3)
[1] 6
```

The 'return' function can be used explicitly to specify the value to be returned by an R function.

```
> p <- function (x) { return (x * x) }

> p(3)
[1] 9
```

Expressions

You can separate multiple expressions in a single line using a semi-colon as shown below:

```
> a <- 1; b <- 2; c <- 3

> a
[1] 1

> b
[1] 2

> c
[1] 3
```

You can also use parenthesis to force evaluate an enclosed expression inside it.

```
> 1 * (3.1415 * 2)
[1] 6.283
```

The curly parenthesis can be used if the expression spans multiple lines and is often seen in a function body.

```
> 1 * {(3.1415
+ * 2)}
[1] 6.283
```

Control structures

R provides a number of loop constructs such as *if*, *repeat*, *while* and *for* for performing iterations. The syntax for the 'if' statement is given below:

```
if (condition) expression1 else expression2
```

For example:

```
> if (TRUE) "TRUE"
[1] "TRUE"

> if (FALSE) "Not applicable" else "FALSE"
[1] "FALSE"
```

You can have multiple 'if else' statements in an 'if' statement, as shown below:

```
> x <- 1
> if (x < 0) {
+   "Less than zero"
```